

```
In [1]: import pandas as pd
        from sklearn.preprocessing import LabelEncoder, StandardScaler

        # Carga limpia del archivo
        df = pd.read_csv('Datos.csv')
```

Introducción

En este proyecto, se pretenden analizar distintos datos obtenidos sobre la obesidad en 3 países diferentes, México, Perú y Colombia. Esta base de datos fue obtenida desde la plataforma Kaggle, los datos presentes en esta fueron obtenidos de un estudio realizado en 2019 por Fabio Mendoza y Alexis De la Hoz. Este estudio se publicó en la revista científica Data In Brief, lo que aporta seguridad y fiabilidad sobre los datos. A parte, el estudio contiene un total de 110 citaciones a fuentes/estudios y 9 referencias, lo que soporta tanto los datos como la clasificación de estos, las variables utilizadas, su significado y magnitudes. Este dataset contiene un total de 2111 registros y 17 columnas.

La problemática escogida es la obesidad, esto debido a su alto porcentaje en la población mexicana. Según Global Obesity Observatory, México se encuentra en el puesto 31 en el ranking de países con obesidad, con un porcentaje de 36.86%. Según la Encuesta Nacional de Salud y Nutrición, realizada desde 2020 a 2023, se obtuvo que la prevalencia nacional de obesidad fue de un 37.1%. En mujeres, el porcentaje de prevalencia de obesidad fue 41%, mientras que en hombres fue un 33%. Estos datos fueron presentados en un estudio, donde también aportaban información significativa sobre la obesidad en México, como la edad de los entrevistados y clasificación por edades teniendo en cuenta su porcentaje de prevalencia de grasa.

Mediante el modelo que se va a realizar en esta investigación se pretende analizar todas las variables disponibles en el dataset, así como analizar sus datos y realizar los cambios que se consideren pertinentes para conseguir el mejor modelo posible. Dicho modelo tendrá como objetivo predecir el IMC de las personas teniendo en cuenta las variables medidas. El IMC se traduce como el índice de masa corporal de las personas. Esta variable se calcula a partir del peso y la estatura de la persona. Un modelo de regresión con múltiples variables es ideal para predecir el IMC debido a que se pueden tener en cuenta las dos variables principales más otro tipo de variables que representan los hábitos y actitudes de las personas y pueden ayudar a descubrir cuáles de estos hábitos son más importantes de lo que parecen y tienen un peso clave a la hora de predecir el IMC de una persona.

```
In [2]: # Cálculo matemático del IMC
        df['BMI'] = df['Weight'] / (df['Height']**2)
```

Exploración y comprensión del conjunto de datos

Para poder entender los datos y los diferentes modelos de regresión que se realizarán, primero se deben entender cada una de las variables presentes en el dataset:

- Gender: equivale al género de la persona. Es una variable de tipo string. Solo hay dos respuestas, masculino o femenino.
- Age: equivale a la edad de la persona. Es una variable numérica de tipo decimal. Se mide en años.
- Height: equivale a la altura de la persona. Es una variable numérica de tipo decimal. Se mide en metros.
- Weight: equivale al peso de la persona. Es una variable numérica de tipo decimal. Se mide en kilogramos.

- **family_history**: pregunta si algún miembro familiar ha padecido o padece de sobrepeso. Es una variable de tipo string. Solo hay dos respuestas, si o no.
- **FAVC**: pregunta si la persona ingiere comida de alta cantidad calórica frecuentemente. Es una variable de tipo string. Solo hay dos respuestas, si o no.
- **FCVC**: pregunta si la persona ingiere vegetales en sus comidas de manera frecuente. Es una variable numérica de tipo decimal. Hay tres respuestas: nunca, a veces, siempre, estas respuestas son representadas con números del 1 al 3.
- **NCP**: pregunta el número de comidas que la persona come en su día a día. Es una variable numérica de tipo decimal.
- **CAEC**: pregunta si la persona consume snacks entre comidas. Es una variable de tipo string. Las respuestas disponibles son: no, a veces, frecuentemente, siempre.
- **SMOKE**: pregunta si la persona fuma. Es una variable de tipo string. Solo hay dos respuestas, si o no.
- **CH2O**: pregunta cuanta agua consume la persona a diario. Es una variable numérica de tipo decimal. Se mide en litros.
- **SCC**: pregunta si la persona monitorea a diario las calorías que consume en sus comidas. Solo hay dos respuestas, si o no.
- **FAF**: pregunta que tan a menudo la persona realiza alguna actividad física. Es una variable numérica de tipo decimal. Se mide en días.
- **TUE**: pregunta que tanto tiempo al día la persona utiliza dispositivos electrónicos como televisión, teléfono, tablets... Es una variable numérica de tipo decimal. Se mide en horas.
- **CALC**: pregunta que tan frecuentemente la persona consume alcohol. Es una variable de tipo string. Las respuestas disponibles son: no, a veces, frecuentemente, siempre.
- **MTRANS**: pregunta que tipo de transporte la persona frecuentemente utiliza. Es una variable de tipo string. Las respuestas disponibles son: automóvil, motocicleta, bicicleta, transporte publico, a pie.

Al observar nuestros datos y como se miden, resaltan todos los datos de tipo numérico, pues no hay ninguno que sea entero, todos contienen decimales. Esto es debido a una técnica llamada SMOTE (Syntethic Minority Over-sampling Technique) que los dos investigadores implementaron. Esta técnica consiste en equilibrar el conjunto de datos mediante la creación de nuevas muestras artificiales para los grupos que contienen menos datos. En vez de duplicar registros, SMOTE selecciona vecinos cercanos con respecto a las características de los registros y traza líneas entre ellos para generar nuevos puntos sintéticos, finalmente los registros se obtienen de dichos puntos. La técnica SMOTE fue utilizada para generar un total de 1613 registros, que representaron el 77% del dataset. Los otros 498 registros fueron obtenidos mediante una encuesta web. Aunque solo el 23% de los datos hayan sido registros reales, SMOTE es una técnica validada en la ciencia de datos. La implementación de esta técnica permite que los modelos de regresión que vamos a crear tengan una cantidad de muestra balanceada.

Preparación y tratamiento de los datos

Ya que conocemos el significado de todos los datos, el tipo de dato que son y como son medidos necesitamos prepreparar los datos y realizar una limpieza para que nuestro modelo pueda obtener el mejor resultado posible. Para esto nos vamos a fijar en 3 problemas comunes en la selección de datos y vamos a estar corrigiéndolos en nuestro dataset.

Nuestro primer problema son los diferentes tipos de variables. Podemos observar que nuestra base de datos contiene tanto variables numéricas como de tipo string. Para nuestro modelo, necesitamos que todas las variables sean numéricas para así poder formar correlaciones, observar colinealidades... Por eso, estaremos utilizando 3 diferentes técnicas para la organización de datos:

- Ordinal Encoding: esta técnica consiste en convertir datos categóricos a datos numéricos donde se le asigna un entero único a cada categoría. Se estará utilizando en variables como CAEC y CALC donde existe un orden lógico de frecuencia. Esto permitirá al modelo entender que la respuesta "Siempre" representa una intensidad mayor a la respuesta "A veces".

```
In [3]: # 1. Limpieza de texto (minúsculas y espacios) para evitar errores
df['CAEC'] = df['CAEC'].astype(str).str.strip().str.lower()
df['CALC'] = df['CALC'].astype(str).str.strip().str.lower()

# 2. Definir diccionario
orden = {'no': 0, 'sometimes': 1, 'frequently': 2, 'always': 3, 'nan': 0}

# 3. Aplicar mapeo
df['CAEC'] = df['CAEC'].map(orden).fillna(0) # fillna(0) por seguridad
df['CALC'] = df['CALC'].map(orden).fillna(0)
```

- One-Hot Encoding: técnica que consiste convertir variables tipo string a un formato binario. Para esto, se crearán columnas binarias independientes para cada tipo de transporte y así evitar que el modelo asuma relaciones matemáticas erróneas de jerarquía o magnitud entre ellos. Ya que si se usara Ordinal Encoding el modelo va a estar sesgado por la numeración impuesta, y la actividad de ir en motocicleta etiquetada con un 2, por ejemplo, tendrá mayor impacto que ir en transporte público, etiquetada con un 1. Con esta técnica el modelo asignará un coeficiente a cada columna y así interpretará mejor que respuesta reduce más el IMC de forma significativa.

```
In [4]: # dtype=int asegura que sean 1 y 0, no True/False
df = pd.get_dummies(df, columns=['MTRANS'], prefix='Transp', dtype=int)
```

- Label Encoding: esta técnica consiste en cambiar los valores de tipo string a valores numéricos enteros. Es una técnica parecida al One-Hot Encoding, pero no se crean diferentes columnas, solo se clasifican los dos tipos de respuesta por orden alfabético y se codifican a binario 0 y 1, indicando presencia o ausencia del atributo. Esta técnica será utilizada en variables como "gender", "family_history", "SMOKE", "FAVC", "SCC" con solo 2 respuestas posibles. En género, si es femenino se representará con un 0, si es masculino con un 1. En las demás variables el no se representará con un 0, el sí con un 1.

```
In [5]: le = LabelEncoder()
cols_binarias = ['Gender', 'family_history', 'FAVC', 'SMOKE', 'SCC']

for col in cols_binarias:
    df[col] = le.fit_transform(df[col].astype(str))
```

El segundo problema que detectamos son la magnitud de los datos. En nuestro dataset podemos observar diferente magnitud de datos dependiendo de las variables. En todas las variables de tipo string, al encodearlas obtendremos un valor de 3 como máximo, y el problema aparece cuando observamos otras variables como la edad. Aunque parezcan valores pequeños a simple vista, teniendo en cuenta como el modelo las analiza, nos damos cuenta de que son variables con bastante más valor numéricamente. El modelo no reacciona igual a un valor de 1 en género, y 30 en edad aunque este último dato se considere pequeño en el rango de edades. Por eso necesitamos estandarizar nuestros datos para evitar que el modelo asigne más importancia basándose en las variables con rangos numéricos muy altos. Esta técnica consiste en centrar los datos en una media de 0 y una desviación estándar de 1. Esto hará que el modelo observe los datos de una forma más equilibrada y así facilitar la interpretación de los coeficientes.

```
In [6]: # Guardamos el Target
y = df['BMI']

# Definimos X eliminando Peso, Altura, La etiqueta de Obesidad y el propio BMI
X = df.drop(['Weight', 'Height', 'BMI', 'Obesity'], axis=1)
```

El tercer problema identificado es la multicolinealidad perfecta y el riesgo de fuga de datos (Data Leakage). Al definir el IMC como nuestra variable objetivo, detectamos que el peso y la estatura presentan una colinealidad absoluta con la predicción, ya que el IMC es una función aritmética directa de ambas. Si se mantuvieran estas variables, el modelo no realizaría una predicción basada en patrones de comportamiento, sino un cálculo matemático simple, anulando la predicción de nuestro modelo. Por ello, se decide eliminar el peso y la estatura del conjunto de entrenamiento. Aunque esta decisión puede resultar en un coeficiente de determinación (R^2) numéricamente menor, es la única vía para obtener un modelo útil que permita entender qué hábitos influyen realmente en el IMC. Por lo que el enfoque del modelo cambia de calculos aritméticos a una inferencia estadística real, permitiendo obtener la importancia real de cada hábito.

```
In [7]: import pandas as pd
from sklearn.preprocessing import StandardScaler

# 1. Definimos La variable objetivo (Target)
y = df['BMI'] # Guardamos el IMC antes de borrar nada

# 2. Eliminamos Las columnas que NO queremos (Peso, Altura, IMC, Obesidad) del DataFrame original
X = df.drop(['Weight', 'Height', 'BMI', 'Obesity'], axis=1)

# 3. AHORA SÍ, Estandarizamos TODO Lo que quedó en X
scaler = StandardScaler()

# 4. Guardamos el resultado en un DataFrame nuevo con Los nombres de Las columnas
X_final_escalado = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
```

Podemos observar una tabla de todos nuestros registros ya con todos los datos tratados y las operaciones para evitar problemas realizadas. Observamos que nuestros registros presentan una escala ya estandarizada donde el 0 representa la media de la población. Por lo que el problema de las magnitudes ha sido completamente resuelto. También observamos que todas las variables de tipo string, ya están transformadas, esto debido a que se realizaron los 3 encodings y se estandarizaron. Por último, observamos que se eliminaron las dos columnas que nos iban a producir colinealidad: la altura y el peso, por lo que nuestro modelo ya está listo para ser entrenado.

Selección de características

Para la selección de características del modelo lineal, aplicaremos el coeficiente de Correlación de Pearson. Esta métrica estadística cuantifica la intensidad y dirección de las relaciones lineales entre las variables explicativas y el IMC. Visualizaremos estos resultados mediante una matriz de correlación (Mapa de Calor). El objetivo es identificar las variables con mayor poder predictivo lineal, descartar aquellas que introducen 'ruido' o confusión al modelo y reducir la dimensionalidad. Esto nos permitirá obtener un modelo más simple, interpretable y eficiente, enfocado únicamente en los datos de mayor relevancia estadística.

```
In [8]: import matplotlib.pyplot as plt
import seaborn as sns
```

```

# 1. Preparamos la tabla completa (X + y)
df_completo = X_final_escalado.copy()
df_completo['IMC'] = y.reset_index(drop=True)

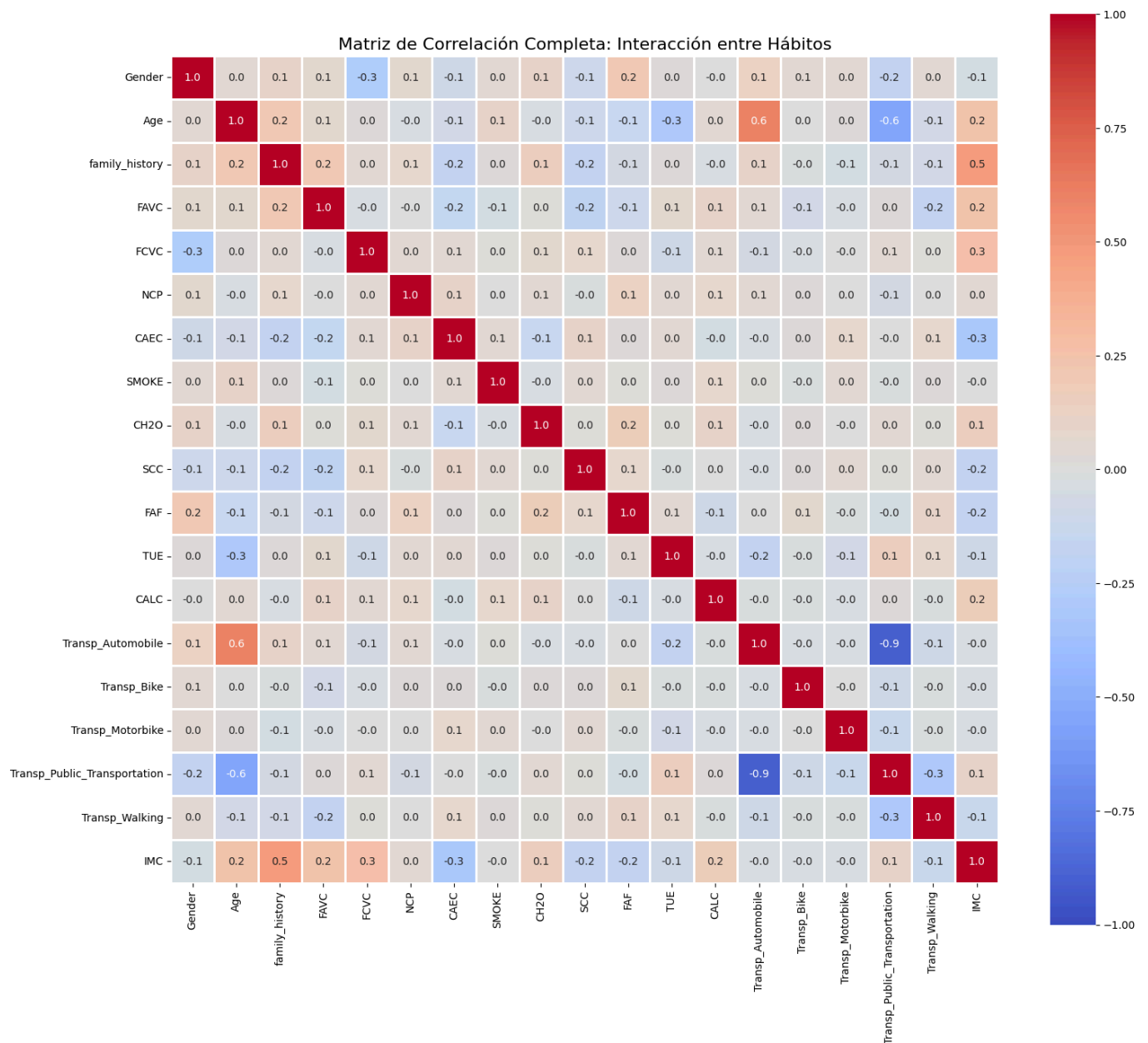
# 2. Calculamos la correlación de TODAS contra TODAS
matriz_completa = df_completo.corr()

# 3. Configuramos el tamaño (tiene que ser grande para que se lean los nombres)
plt.figure(figsize=(16, 14))

# 4. Generamos el Heatmap
sns.heatmap(matriz_completa,
            annot=True,          # Ponemos los números
            fmt=".1f",          # Usamos solo 1 decimal para que no se amontone
            cmap='coolwarm',    # Rojo/Azul
            vmin=-1, vmax=1,    # Escala
            linewidths=1,       # Bordes blancos (cuadritos)
            linecolor='white',   # Celdas cuadradas perfectas
            square=True)

plt.title('Matriz de Correlación Completa: Interacción entre Hábitos', fontsize=16)
plt.tight_layout()
plt.show()

```



Aquí podemos observar la matriz de correlación completa. La conclusión obtenida al visualizar las correlaciones es que todas las variables son independientes, no hay ninguna que se acerque completamente a otra y muestre colinealidad. Para nuestro modelo, nos centramos en las variables que tienen mayor correlación con el IMC, pues estas variables representan a los hábitos más relevantes para realizar la predicción. Elegimos las siguientes variables:

- family_history (historial de familiares con obesidad)
- CAEC (consumición de snacks entre comidas)
- FAVC (alimentación de comida con altas cantidades calóricas)
- Age (edad)
- SCC (monitoreo de calorías diario)

Es importante recalcar que elegimos las variables con mayor correlación sin ver el signo, el signo solo indica si esta variable aumenta o disminuye el IMC. Se busca que impacte el IMC, no si impacta para bien o para mal.

Para nuestro modelo no lineal, estaremos utilizando una técnica llamada Información Mutua. Esta técnica nos permite detectar cualquier tipo de dependencia entre los hábitos y el IMC, midiendo que tanto se reduce la incertidumbre sobre el IMC conociendo el valor de una variable específica. Si utilizáramos la misma técnica, la Correlación Pearson, podríamos perdernos de conexiones entre variables importantes a la hora de elegir dichas variables. A parte, la técnica de Correlación Pearson es una técnica lineal, y estamos seleccionando variables para nuestro modelo no lineal.

```
In [9]: from sklearn.feature_selection import mutual_info_regression
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Calcular La Información Mutua (Mutual Information)
# Esta función analiza relaciones complejas, no solo líneas rectas
mi_scores = mutual_info_regression(X_final_escalado, y, random_state=42)

# 2. Organizar Los datos para graficar
mi_series = pd.Series(mi_scores, index=X_final_escalado.columns)
mi_series = mi_series.sort_values(ascending=False) # Ordenar de mayor a menor

# 3. Graficar el Ranking
plt.figure(figsize=(10, 8))
sns.barplot(x=mi_series.values, y=mi_series.index, palette='viridis')

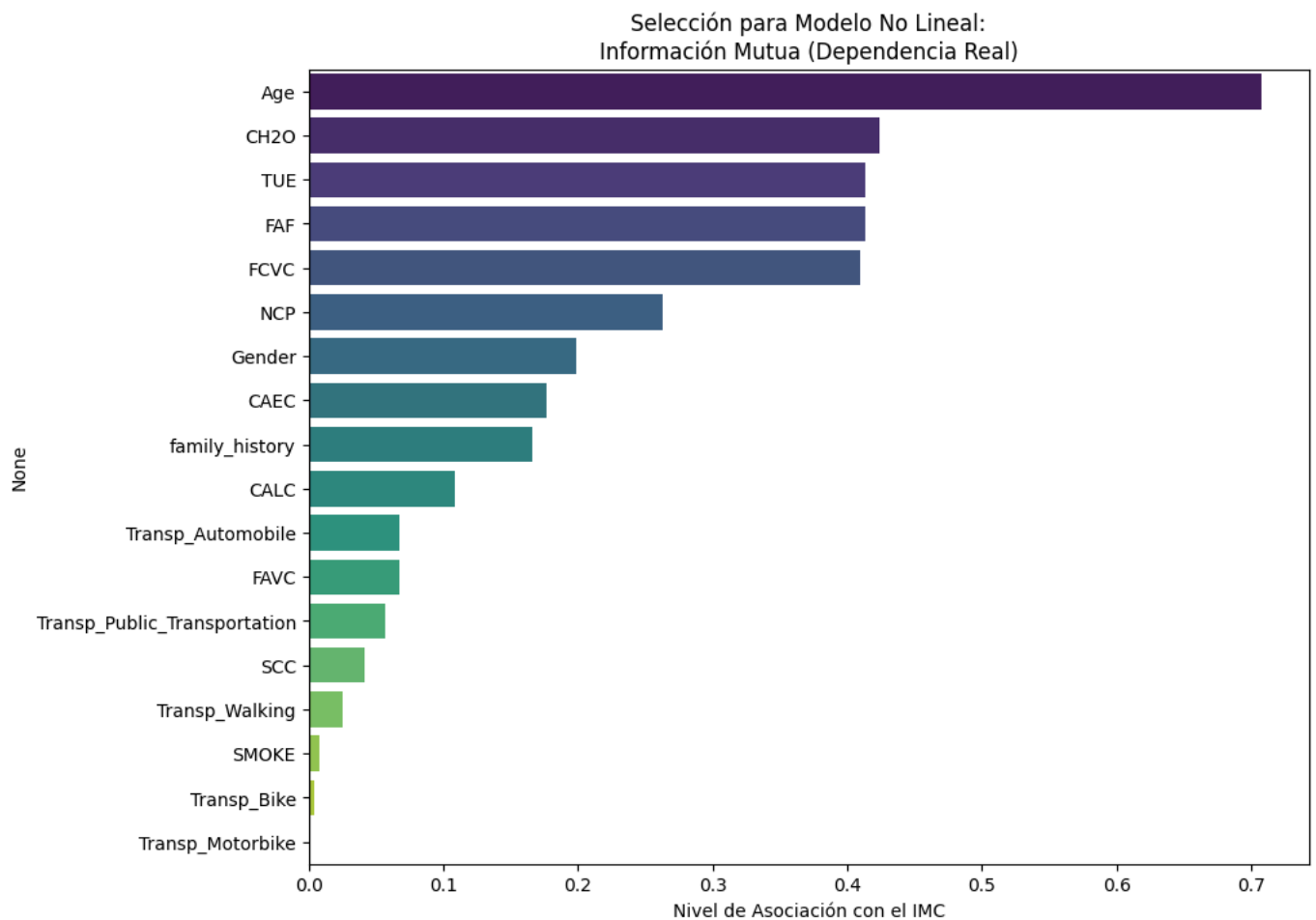
plt.title('Selección para Modelo No Lineal:\nInformación Mutua (Dependencia Real)')
plt.xlabel('Nivel de Asociación con el IMC')
plt.show()

# 4. Imprimir Los valores numéricos para decidir el corte
print("--- Ranking de Importancia (Mutual Info) ---")
print(mi_series)
```

/var/folders/px/dlp0zrrn4w16nb0rn551ywd80000gn/T/ipykernel_25810/1593346444.py:16: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=mi_series.values, y=mi_series.index, palette='viridis')
```



--- Ranking de Importancia (Mutual Info) ---

Age	0.707262
CH20	0.423555
TUE	0.413682
FAF	0.413509
FCVC	0.409104
NCP	0.262917
Gender	0.198662
CAEC	0.177037
family_history	0.166547
CALC	0.108222
Transp_Automobile	0.067296
FAVC	0.067202
Transp_Public_Transportation	0.056543
SCC	0.041550
Transp_Walking	0.024865
SMOKE	0.007780
Transp_Bike	0.004090
Transp_Motorbike	0.001281

dtype: float64

Con esta técnica podemos observar resultados bastante distintos a los observados con la técnica Pearson, esto debido a que si las correlaciones son generadas mediante curvas, la técnica Pearson al ser lineal las va a desechar, mientras que la técnica de Información Mutua las tiene en cuenta y las podemos interpretar. Basándonos en nuestra gráfica, seleccionaremos las siguientes variables:

- Age (edad)
- CH20 (consumo en litros de agua)
- TUE (grado de uso de dispositivos electrónicos a diario)
- FAF (que tan a menudo la persona realiza actividad física)
- FCVC (consumición de vegetales en las comidas)

Comparando las variables seleccionadas, observamos que la variable Age es la única que se repite, aunque la segunda herramienta nos dice que tiene mucha más importancia de lo que nos da a entender la Correlación Pearson. También otro cambio notorio es la variable family_history, pues en esta última tabla se encuentra justo a la mitad, y en la Correlación Pearson era la variable más alta. Esto puede ser debido a la forma de actuar de las dos técnicas, donde se observan diferencias con las variables de comportamiento continuo que aportan mas información que las variables booleanas.

Observando los resultados, concluimos que incluir variables irrelevantes habría aumentado la complejidad computacional y el riesgo de sobreajuste, haciendo que el modelo memorizara datos inútiles en lugar de aprender patrones reales. Por otro lado, haber eliminado la redundancia mediante la eliminación de las variables altura y peso garantiza que nuestros modelos sean estables y que los coeficientes sean interpretables, permitiéndonos aislar el efecto real de cada hábito sobre la obesidad.

Construcción y comparación de modelos

Teniendo seleccionadas las variables para ambos modelos, procedemos a la creación de estos. Estaremos entrenando a dos modelos de regresión, uno lineal y otro no y finalmente comparando los dos para ver cual nos ayudará a obtener mejor predicción. Comenzaremos con el modelo de regresión lineal, para esto desarrollamos un modelo de regresión lineal multiple con el objetivo de establecer una linea base de comparación y observar si realmente existe una relación entre los hábitos de vida y el IMC que se pueda explicar mediante una tendencia lineal directa. Este enfoque es muy útil para identificar el impacto individual de cada una de las variables que seleccionamos a través de sus coeficientes.

```
In [10]: # =====
# MODELO DE REGRESIÓN LINEAL MÚLTIPLE
# Objetivo : Predecir IMC (BMI) – Interpretabilidad de Betas
# Variables : Age, family_history_with_overweight, FAVC,
#             CAEC, SCC (selección por Correlación de Pearson)
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.preprocessing import LabelEncoder, StandardScaler

import warnings
warnings.filterwarnings('ignore')

# -----
# BLOQUE 1 · CARGA DEL DATASET
# -----

df = pd.read_csv("Datos.csv")

# Calcular BMI si no existe
if 'IMC' not in df.columns:
    df['IMC'] = df['Weight'] / (df['Height'] ** 2)
    print("✓ Columna IMC calculada y añadida.")

print(f"Dataset cargado: {df.shape[0]} filas × {df.shape[1]} columnas")
df.head()
```



```

# -----
# BLOQUE 2 · PREPROCESAMIENTO
# -----

# 2.1 Eliminar variables que causan data leakage
cols_excluir = ['Weight', 'Height', 'NObeyesdad']
df_model = df.drop(columns=[c for c in cols_excluir if c in df.columns])

# 2.2 Codificar variables categóricas
le = LabelEncoder()
for col in df_model.select_dtypes(include='object').columns:
    df_model[col] = le.fit_transform(df_model[col].astype(str))

# 2.3 Separar target y features
y = df_model['IMC']
X_full = df_model.drop(columns=['IMC'])

# 2.4 Escalar con StandardScaler
# (escalar ANTES de seleccionar features es la práctica correcta)
scaler = StandardScaler()
X_scaled = pd.DataFrame(
    scaler.fit_transform(X_full),
    columns=X_full.columns
)

# 2.5 Selección flexible de columnas (tolera sufijos de dummificación)
# Ej: 'CAEC' captura también 'CAEC_1', 'CAEC_2', etc.
FEATURES_PEARSON = [
    'Age',
    'family_history',
    'FAVC',
    'CAEC',
    'SCC'
]

def seleccionar_columnas(df, nombres):
    seleccionadas = []
    for nombre in nombres:
        coincidencias = [
            c for c in df.columns
            if c == nombre or c.startswith(nombre + '_')
        ]
        seleccionadas.extend(coincidencias)
    return seleccionadas

cols_lineal = seleccionar_columnas(X_scaled, FEATURES_PEARSON)
X_lineal = X_scaled[cols_lineal]

print(f"\n✓ Variables del modelo lineal ({len(cols_lineal)}):")
print(cols_lineal)

# -----
# BLOQUE 3 · DIVISIÓN TRAIN / TEST
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X_lineal, y,
    test_size = 0.20,
    random_state = 42
)

```

```

print(f"\n✓ Entrenamiento : {X_train.shape}")
print(f"✓ Prueba          : {X_test.shape}")

# -----
# BLOQUE 4 · ENTRENAMIENTO DEL MODELO
# -----

modelo_lr = LinearRegression()
modelo_lr.fit(X_train, y_train)

print("\n✓ Modelo de Regresión Lineal entrenado correctamente.")

# -----
# BLOQUE 5 · INTERPRETACIÓN DE COEFICIENTES (BETAS)
# -----

# DataFrame con intercepto + coeficientes ordenados por impacto absoluto
coeficientes = pd.DataFrame({
    'Variable'      : ['Intercepto ( $\beta_0$ )'] + cols_lineal,
    'Coeficiente'   : [modelo_lr.intercept_] + list(modelo_lr.coef_)
})

coeficientes['Impacto'] = coeficientes['Coeficiente'].apply(
    lambda x: '↑ Aumenta IMC' if x > 0 else '↓ Reduce IMC'
)

# Separar intercepto del resto para ordenar solo los betas
intercepto_row = coeficientes[coeficientes['Variable'] == 'Intercepto ( $\beta_0$ )']
betas_df       = coeficientes[coeficientes['Variable'] != 'Intercepto ( $\beta_0$ )'] \
    .sort_values('Coeficiente', ascending=False) \
    .reset_index(drop=True)

print("\n" + "="*58)
print("  TABLA DE COEFICIENTES – Regresión Lineal")
print("="*58)
print(intercepto_row[['Variable', 'Coeficiente']].to_string(index=False))
print("-"*58)
print(betas_df[['Variable', 'Coeficiente', 'Impacto']].to_string(index=False))
print("="*58)
print("\nInterpretación: Coeficientes estandarizados.")
print("Un  $\beta$  de +1.5 significa que al aumentar esa variable")
print("1 desviación estándar, el IMC sube 1.5 kg/m2.")

# -----
# BLOQUE 6 · MÉTRICAS DE EVALUACIÓN (Train vs Test)
# -----

y_pred_train = modelo_lr.predict(X_train)
y_pred       = modelo_lr.predict(X_test)
residuos     = y_test - y_pred

r2_train     = r2_score(y_train, y_pred_train)
r2_test      = r2_score(y_test, y_pred)
mae          = mean_absolute_error(y_test, y_pred)
diferencia   = r2_train - r2_test

print("\n" + "="*45)
print("  RESULTADOS – Regresión Lineal Múltiple")
print("="*45)
print(f"    R2 Train          : {r2_train:.4f}")

```

```

print(f"    R² Test                : {r2_test:.4f}")
print(f"    Diferencia (Train-Test): {diferencia:.4f}")
print(f"    MAE (Error Abs. Medio): {mae:.4f} kg/m²")
print("="*45)

# En regresión lineal una diferencia pequeña es lo esperado
if diferencia < 0.05:
    estado = "Sin overfitting ✓ – modelo estable"
elif diferencia < 0.15:
    estado = "Overfitting leve ⚠️ – aceptable"
else:
    estado = "Overfitting alto ✗ – revisar features"

print(f"    Overfitting                : {estado}")

# Gráfica comparativa R² Train vs Test
fig, ax = plt.subplots(figsize=(5, 4))

barras = ax.bar(
    ['R² Train', 'R² Test'],
    [r2_train, r2_test],
    color=['#00e5a0', '#4f8eff'],
    edgecolor='white', linewidth=0.5, width=0.4
)

for bar, val in zip(barras, [r2_train, r2_test]):
    ax.text(bar.get_x() + bar.get_width()/2, val + 0.01,
            f"{val:.4f}", ha='center', fontsize=11,
            fontweight='bold', color='white')

ax.set_ylim(0, 1.1)
ax.set_title("Comparativa R² – Train vs Test\nRegresión Lineal Múltiple",
            fontsize=12, fontweight='bold', pad=12)
ax.set_ylabel("R² Score", fontsize=10)
ax.axhline(y=1.0, color='white', linestyle=':', alpha=0.3)
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig("lineal_r2_train_vs_test.png", dpi=150, bbox_inches='tight')
plt.show()
print("✓ Gráfica guardada → lineal_r2_train_vs_test.png")

# Reasignar r2 para que los bloques siguientes lo usen correctamente
r2 = r2_test

# -----
# BLOQUE 7 · VISUALIZACIÓN 1 – Gráfica de Coeficientes
# -----

# Color por signo: azul = impacto positivo, rojo = impacto negativo
colores = ['#4f8eff' if v > 0 else '#ff6b6b' for v in betas_df['Coeficiente']]

fig, ax = plt.subplots(figsize=(8, 4))

bars = ax.barh(
    betas_df['Variable'],
    betas_df['Coeficiente'],
    color = colores,
    edgecolor = 'white',
    linewidth = 0.5,
    height = 0.55
)

```

```

# Línea de referencia en cero
ax.axvline(x=0, color='white', linewidth=1.2, linestyle='--', alpha=0.6)

# Etiqueta con valor exacto en cada barra
for bar, val in zip(bars, betas_df['Coeficiente']):
    offset = 0.03 if val >= 0 else -0.03
    align = 'left' if val >= 0 else 'right'
    ax.text(
        val + offset,
        bar.get_y() + bar.get_height() / 2,
        f"{val:+.3f}",
        va='center', ha=align,
        fontsize=10, color='white', fontweight='bold'
    )

# Leyenda manual
from matplotlib.patches import Patch
leyenda = [
    Patch(facecolor='#4f8eff', label='↑ Aumenta el BMI'),
    Patch(facecolor='#ff6b6b', label='↓ Reduce el BMI'),
]
ax.legend(handles=leyenda, fontsize=9, loc='lower right')

ax.set_title(
    "Coeficientes  $\beta$  - Regresión Lineal (IMC)\n"
    "Variables estandarizadas · Pearson Top-5",
    fontsize=12, fontweight='bold', pad=12
)
ax.set_xlabel("Coeficiente  $\beta$  (impacto por 1 desv. estándar)", fontsize=10)
ax.set_ylabel("")
ax.grid(axis='x', alpha=0.3)
ax.invert_yaxis() # la variable de mayor impacto arriba

plt.tight_layout()
plt.savefig("coeficientes_regresion_lineal.png", dpi=150, bbox_inches='tight')
plt.show()
print("✓ Gráfica guardada → coeficientes_regresion_lineal.png")

# -----
# BLOQUE 8 · VISUALIZACIÓN 2 – Gráfica de Residuos
# -----
#
# Un buen modelo lineal debe mostrar:
# · Residuos distribuidos aleatoriamente alrededor de 0
# · Sin patrones en forma de abanico o curva (homocedasticidad)

fig, ax = plt.subplots(figsize=(7, 5))

ax.scatter(
    y_pred, residuos,
    alpha = 0.45,
    s = 30,
    color = '#a78bfa',
    edgecolors = 'white',
    linewidths = 0.3,
    label = 'Residuos'
)

# Línea de residuo = 0 (referencia ideal)
ax.axhline(y=0, color='#00e5a0', linewidth=2,
           linestyle='--', label='Residuo = 0 (ideal)')

```

```

# Banda ±MAE como referencia de error típico
ax.axhline(y= mae, color='salmon', linewidth=1,
           linestyle=':', alpha=0.7, label=f'+MAE ({mae:.2f})')
ax.axhline(y=-mae, color='salmon', linewidth=1,
           linestyle=':', alpha=0.7, label=f'-MAE ({mae:.2f})')

# Anotación de R² dentro del gráfico
ax.text(
    0.03, 0.95, f"R² = {r2:.4f}",
    transform=ax.transAxes,
    fontsize=10, verticalalignment='top',
    bbox=dict(boxstyle='round,pad=0.4',
              facecolor='#1a1d2e', edgecolor='#a78bfa', alpha=0.9)
)

ax.set_title(
    "Residuos vs. Predicciones – Regresión Lineal\n"
    "Verificación de homocedasticidad",
    fontsize=12, fontweight='bold', pad=12
)

ax.set_xlabel("IMC Predicho (kg/m²)", fontsize=10)
ax.set_ylabel("Residuo (Real - Predicho)", fontsize=10)
ax.legend(fontsize=9)
ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig("residuos_regresion_lineal.png", dpi=150, bbox_inches='tight')
plt.show()
print("✓ Gráfica guardada → residuos_regresion_lineal.png")

# -----
# BLOQUE 9 · RESUMEN FINAL
# -----

print("\n" + "="*50)
print(" RESUMEN DEL PIPELINE – Regresión Lineal")
print("="*50)
print(f" Modelo : LinearRegression (OLS)")
print(f" Features : {cols_lineal}")
print(f" Train / Test: 80% / 20% (random_state=42)")
print(f" Intercepto : {modelo_lr.intercept_:.4f}")
print(f" R² (test) : {r2:.4f}")
print(f" MAE (test) : {mae:.4f} kg/m²")
print("="*50)

```

✓ Columna IMC calculada y añadida.
Dataset cargado: 2111 filas × 18 columnas

✓ Variables del modelo lineal (5):
['Age', 'family_history', 'FAVC', 'CAEC', 'SCC']

✓ Entrenamiento : (1688, 5)

✓ Prueba : (423, 5)

✓ Modelo de Regresión Lineal entrenado correctamente.

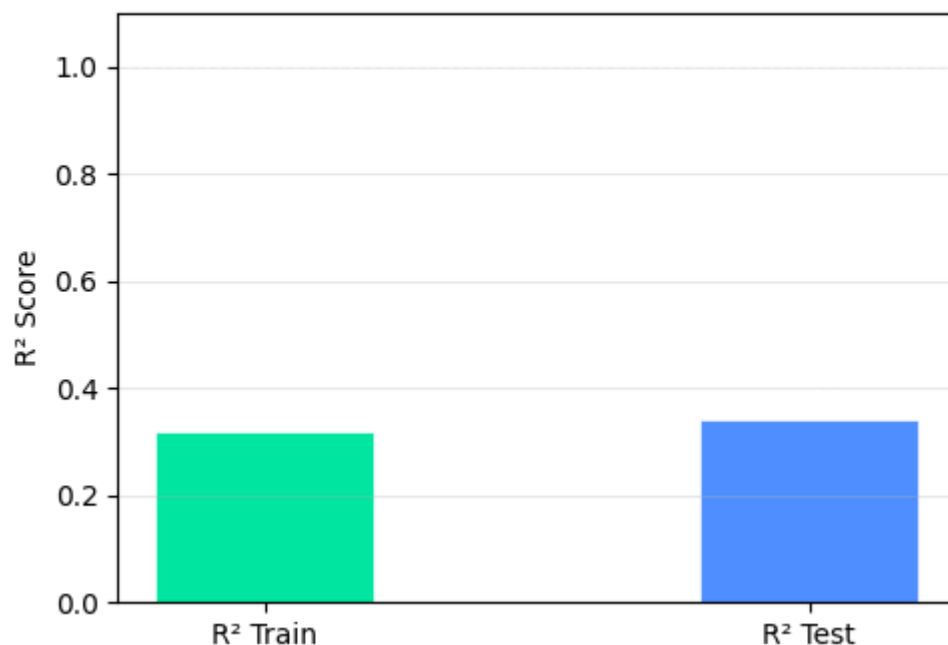
TABLA DE COEFICIENTES – Regresión Lineal		
Variable	Coeficiente	
Intercepto (β_0)	29.671781	
Variable	Coeficiente	Impacto
family_history	3.086140	↑ Aumenta IMC
CAEC	1.639647	↑ Aumenta IMC
Age	1.035751	↑ Aumenta IMC
FAVC	0.833995	↑ Aumenta IMC
SCC	-0.447712	↓ Reduce IMC

Interpretación: Coeficientes estandarizados.

Un β de +1.5 significa que al aumentar esa variable 1 desviación estándar, el IMC sube 1.5 kg/m².

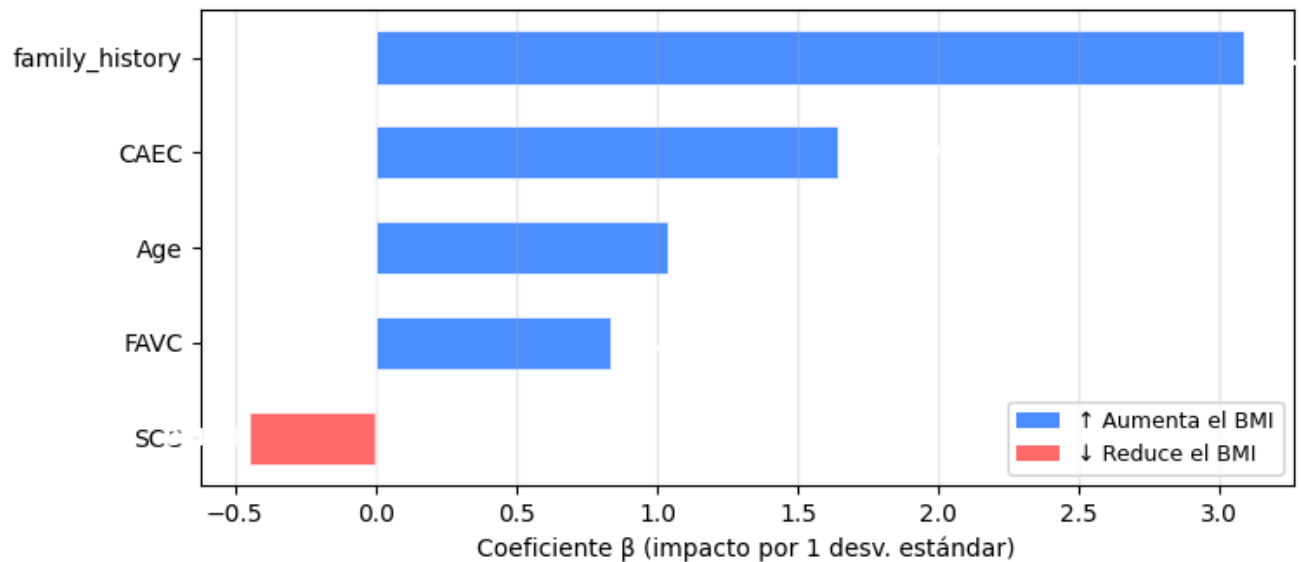
RESULTADOS – Regresión Lineal Múltiple	
R ² Train	: 0.3192
R ² Test	: 0.3419
Diferencia (Train-Test):	-0.0228
MAE (Error Abs. Medio):	5.2407 kg/m ²
Overfitting	: Sin overfitting ✓ – modelo estable

Comparativa R² – Train vs Test Regresión Lineal Múltiple



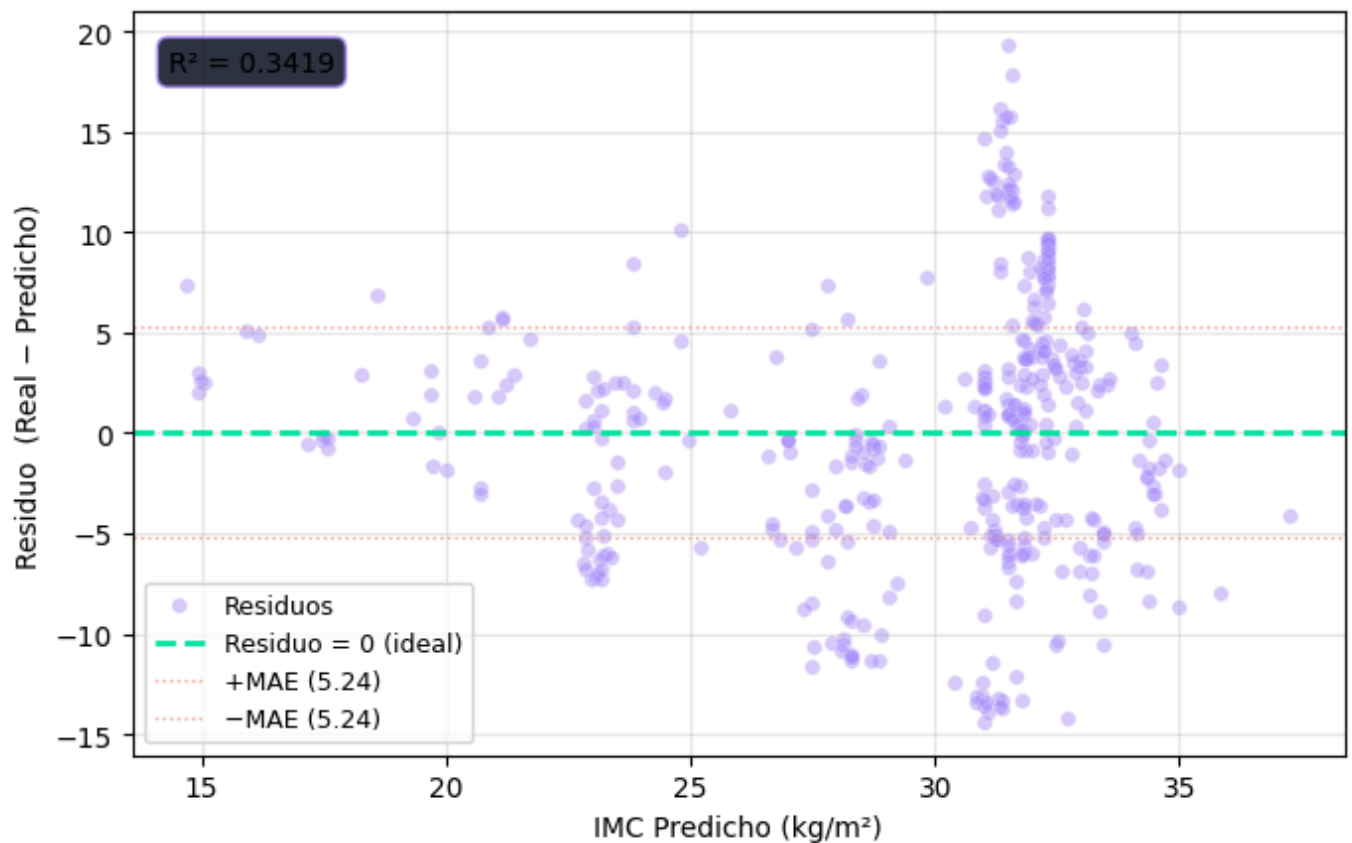
✓ Gráfica guardada → lineal_r2_train_vs_test.png

Coeficientes β — Regresión Lineal (IMC) Variables estandarizadas · Pearson Top-5



✓ Gráfica guardada → coeficientes_regresion_lineal.png

Residuos vs. Predicciones — Regresión Lineal Verificación de homocedasticidad



✓ Gráfica guardada → residuos_regresion_lineal.png

```
=====
RESUMEN DEL PIPELINE — Regresión Lineal
=====
Modelo      : LinearRegression (OLS)
Features    : ['Age', 'family_history', 'FAVC', 'CAEC', 'SCC']
Train / Test: 80% / 20% (random_state=42)
Intercepto  : 29.6718
R² (test)   : 0.3419
MAE (test)  : 5.2407 kg/m²
=====
```

Primero generamos una tabla de coeficientes con una gráfica que sirve como su representación en donde observamos cuanto impacto tiene cada variable. La variable con más impacto es "family_history", con un coeficiente casi el doble de mayor con respecto a la variable "CAEC". Esto puede indicar una relación con la genética, donde tiene gran impacto en el índice de masa corporal de la persona. También observamos que la variable que tiene coeficiente negativo y por lo tanto disminuye el índice de masa corporal es la variable "SCC". Esta variable corresponde a la medición de calorías diario, donde se puede relacionar con gente que va al gimnasio o realiza algún tipo de dieta.

Posteriormente, observamos nuestra gráfica de verificación de homocedasticidad. Está gráfica consiste en comparar el error cometido frente al valor estimado por el modelo. Observamos que los residuos no se distribuyen de forma aleatoria, sino que se van alejando a medida que el IMC aumenta. Al evaluar el desempeño de nuestro modelo de Regresión Lineal Múltiple, comparamos su capacidad explicativa entre los datos de entrenamiento (Train) y los datos de validación (Test). Observamos un R^2 de 0.3192 en el entrenamiento y de 0.3419 en la prueba. Esta diferencia mínima de -0.02 evidencia un escenario de subajuste leve (underfitting).

El hecho de que los puntajes sean consistentes (e incluso ligeramente mejores en la fase de prueba) demuestra que el modelo no memoriza ruido y es altamente estable frente a datos nuevos. Sin embargo, los valores globales bajos suceden porque la estructura paramétrica del modelo lineal asume una relación recta entre las variables, lo cual resulta insuficiente para capturar toda la complejidad multicausal del IMC. Esta limitación en su capacidad predictiva y su margen de error (MAE) de 5.24 kg/m² confirman matemáticamente nuestra principal conclusión metodológica: la regresión lineal es una herramienta excelente y estable para la inferencia, pero carece de la flexibilidad matemática necesaria para realizar predicciones exactas, justificando nuestra transición hacia modelos no lineales.

```
In [14]: # =====
#  MODELO K-NEAREST NEIGHBORS (KNN) REGRESSOR
#  Objetivo : Predecir IMC (BMI) – Dataset de Obesidad
#  Variables : Age, CH20, TUE, FAF, FCVC (selección por MI)
#  =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import r2_score, mean_absolute_error

import warnings
warnings.filterwarnings('ignore')

# -----
# BLOQUE 1 · CARGA Y PREPROCESAMIENTO
# -----

df = pd.read_csv("Datos.csv")

if 'IMC' not in df.columns:
    df['IMC'] = df['Weight'] / (df['Height'] ** 2)
    print("✓ Columna IMC calculada.")

# Eliminar Leakage
```



```

cols_excluir = ['Weight', 'Height', 'NObeyesdad']
df_model = df.drop(columns=[c for c in cols_excluir if c in df.columns])

# Codificar categóricas
le = LabelEncoder()
for col in df_model.select_dtypes(include='object').columns:
    df_model[col] = le.fit_transform(df_model[col].astype(str))

# Target y features
y = df_model['IMC'].reset_index(drop=True)
X_full = df_model.drop(columns=['IMC'])

# Escalar - KNN es MUY sensible a la escala, esto es crítico
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X_full), columns=X_full.columns)

# Selección flexible de columnas (tolera sufijos de dummificación)
FEATURES_MI = ['Age', 'CH20', 'TUE', 'FAF', 'FCVC']

def seleccionar_columnas(df, nombres):
    sel = []
    for n in nombres:
        sel.extend([c for c in df.columns if c == n or c.startswith(n + '_')])
    return sel

cols = seleccionar_columnas(X_scaled, FEATURES_MI)
X = X_scaled[cols].reset_index(drop=True)

print(f"✓ Features usadas : {cols}")
print(f"✓ Shape X: {X.shape} | Shape y: {y.shape}")

# -----
# BLOQUE 2 · DIVISIÓN TRAIN / TEST
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

print(f"\n✓ Train: {X_train.shape} | Test: {X_test.shape}")

# -----
# BLOQUE 3 · BÚSQUEDA DEL K ÓPTIMO
# -----
# Probamos k de 1 a 30 y graficamos el R² para elegir el mejor k

k_valores = range(1, 31)
r2_por_k = []

for k in k_valores:
    knn_temp = KNeighborsRegressor(n_neighbors=k, weights='distance')
    knn_temp.fit(X_train, y_train)
    r2_por_k.append(r2_score(y_test, knn_temp.predict(X_test)))

k_optimo = k_valores[np.argmax(r2_por_k)]
print(f"\n✓ K óptimo encontrado: k = {k_optimo} (R² = {max(r2_por_k):.4f})")

# Gráfica de selección de k
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(k_valores, r2_por_k, color='#4f8eff', linewidth=2, marker='o',
        markersize=4, markerfacecolor='white', markeredgewidth=1.5)

```

```

ax.axvline(x=k_optimo, color='#00e5a0', linewidth=2,
           linestyle='--', label=f'K óptimo = {k_optimo}')
ax.scatter([k_optimo], [max(r2_por_k)], color='#00e5a0', s=100, zorder=5)
ax.set_title("Selección del K óptimo – KNN Regressor", fontsize=12, fontweight='bold')
ax.set_xlabel("Número de vecinos (k)", fontsize=10)
ax.set_ylabel("R² Score", fontsize=10)
ax.legend(fontsize=10)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("knn_seleccion_k.png", dpi=150, bbox_inches='tight')
plt.show()
print("✓ Gráfica guardada → knn_seleccion_k.png")

```

```

# -----
# BLOQUE 4 · ENTRENAMIENTO CON K ÓPTIMO
# -----

modelo_knn = KNeighborsRegressor(
    n_neighbors = k_optimo,
    weights      = 'distance', # vecinos más cercanos pesan más
    metric       = 'euclidean'
)

modelo_knn.fit(X_train, y_train)
y_pred = modelo_knn.predict(X_test)

print(f"\n✓ Modelo KNN entrenado con k = {k_optimo}.")

# -----
# BLOQUE 5 · MÉTRICAS DE EVALUACIÓN
# -----

y_pred_train = modelo_knn.predict(X_train)
y_pred        = modelo_knn.predict(X_test)

r2_train = r2_score(y_train, y_pred_train)
r2_test  = r2_score(y_test, y_pred)
mae       = mean_absolute_error(y_test, y_pred)
diferencia = r2_train - r2_test

print("\n" + "="*45)
print("  RESULTADOS – KNN Regressor")
print("="*45)
print(f"    K vecinos           : {k_optimo}")
print(f"    R² Train            : {r2_train:.4f}")
print(f"    R² Test             : {r2_test:.4f}")
print(f"    Diferencia (Train-Test): {diferencia:.4f}")
print(f"    MAE (Error Abs. Medio): {mae:.4f} kg/m²")
print("="*45)

# Interpretación de overfitting
# Nota: KNN con weights='distance' y k=1 da R² Train = 1.0 siempre,
# por eso la diferencia suele ser más alta que en otros modelos.
if diferencia < 0.05:
    estado = "Sin overfitting ✓ – el modelo generaliza bien"
elif diferencia < 0.15:
    estado = "Overfitting leve ⚠ – aceptable pero vigilar"
else:
    estado = "Overfitting alto ✗ – prueba un k mayor"

print(f"    Overfitting           : {estado}")

```

```

# Gráfica comparativa R² Train vs Test
fig, ax = plt.subplots(figsize=(5, 4))

barras = ax.bar(
    ['R² Train', 'R² Test'],
    [r2_train, r2_test],
    color=['#00e5a0', '#4f8eff'],
    edgecolor='white', linewidth=0.5, width=0.4
)

for bar, val in zip(barras, [r2_train, r2_test]):
    ax.text(bar.get_x() + bar.get_width()/2, val + 0.01,
            f"{val:.4f}", ha='center', fontsize=11,
            fontweight='bold', color='white')

ax.set_ylim(0, 1.1)
ax.set_title(f"Comparativa R² - Train vs Test\nKNN Regressor (k={k_optimo})",
            fontsize=12, fontweight='bold', pad=12)
ax.set_ylabel("R² Score", fontsize=10)
ax.axhline(y=1.0, color='white', linestyle=':', alpha=0.3)
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig("knn_r2_train_vs_test.png", dpi=150, bbox_inches='tight')
plt.show()
print("✓ Gráfica guardada → knn_r2_train_vs_test.png")

# Reasignar r2 para que el Bloque 6 lo use correctamente
r2 = r2_test

# -----
# BLOQUE 6 · VISUALIZACIÓN – Real vs. Predicho
# -----

fig, ax = plt.subplots(figsize=(7, 6))

ax.scatter(
    y_test, y_pred,
    alpha=0.45, s=35,
    color='#4f8eff', edgecolors='white', linewidths=0.3,
    label='Predicciones KNN'
)

# Línea de referencia 45°
lim = [
    min(y_test.min(), y_pred.min()) - 1,
    max(y_test.max(), y_pred.max()) + 1
]
ax.plot(lim, lim, color='#00e5a0', linewidth=2,
        linestyle='--', label='Predicción perfecta (45°)')

# Métricas anotadas
ax.text(
    0.04, 0.93,
    f"K = {k_optimo}\nR² = {r2:.4f}\nMAE = {mae:.4f} kg/m²",
    transform=ax.transAxes, fontsize=10, verticalalignment='top',
    bbox=dict(boxstyle='round,pad=0.5',
              facecolor='#1a1d2e', edgecolor='#4f8eff', alpha=0.9)
)

ax.set_xlim(lim); ax.set_ylim(lim)
ax.set_title(f"KNN Regressor (k={k_optimo}) – Real vs. Predicho",

```

```

        fontsize=12, fontweight='bold', pad=12)
ax.set_xlabel("IMC Real (kg/m²)",      fontsize=10)
ax.set_ylabel("IMC Predicho (kg/m²)",  fontsize=10)
ax.legend(fontsize=9)
ax.grid(alpha=0.3)
ax.set_aspect('equal')

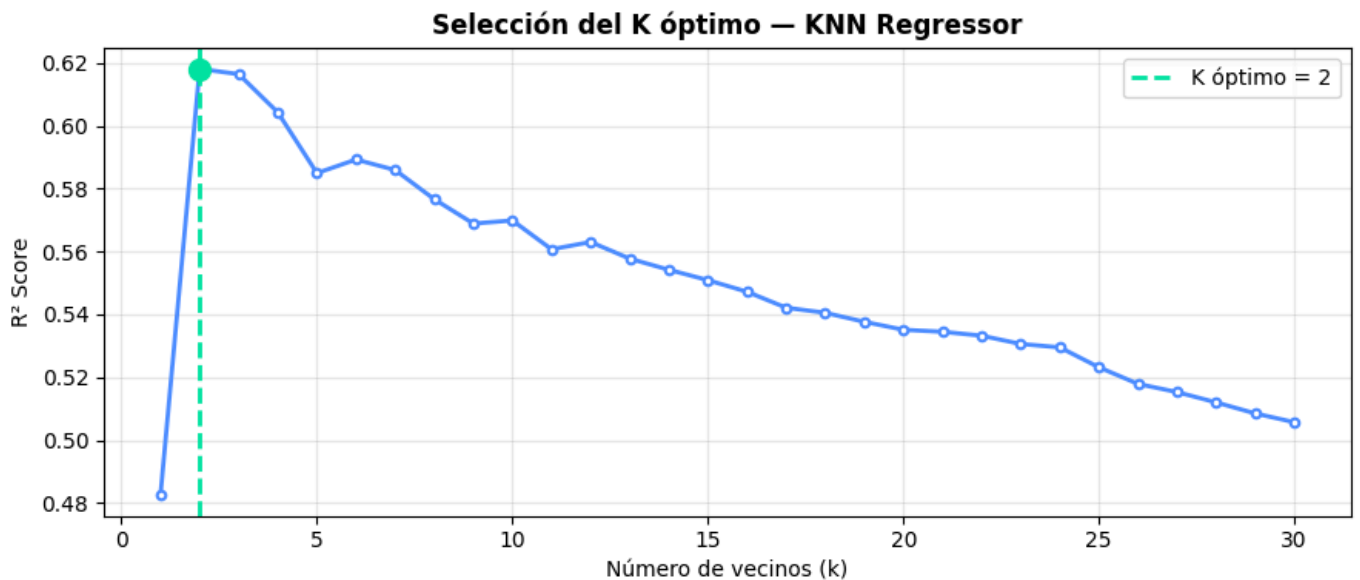
plt.tight_layout()
plt.savefig("knn_real_vs_predicho.png", dpi=150, bbox_inches='tight')
plt.show()
print("✓ Gráfica guardada → knn_real_vs_predicho.png")

# -----
# BLOQUE 7 · RESUMEN FINAL
# -----

print("\n" + "="*45)
print("  RESUMEN – KNN Regressor")
print("="*45)
print(f"  Modelo      : KNeighborsRegressor")
print(f"  K óptimo    : {k_optimo} (pesos por distancia)")
print(f"  Features    : {cols}")
print(f"  Train / Test: 80% / 20% (random_state=42)")
print(f"  R² (test)   : {r2:.4f}")
print(f"  MAE (test)  : {mae:.4f} kg/m²")
print("="*45)

```

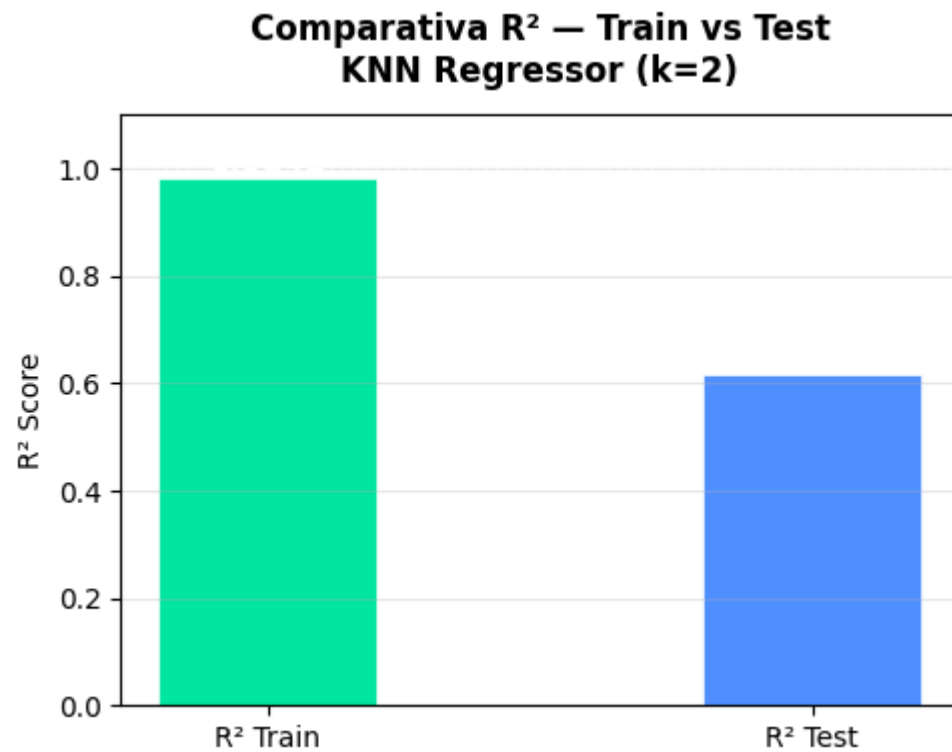
- ✓ Columna IMC calculada.
- ✓ Features usadas : ['Age', 'CH20', 'TUE', 'FAF', 'FCVC']
- ✓ Shape X: (2111, 5) | Shape y: (2111,)
- ✓ Train: (1688, 5) | Test: (423, 5)
- ✓ K óptimo encontrado: k = 2 ($R^2 = 0.6181$)



✓ Gráfica guardada → knn_seleccion_k.png

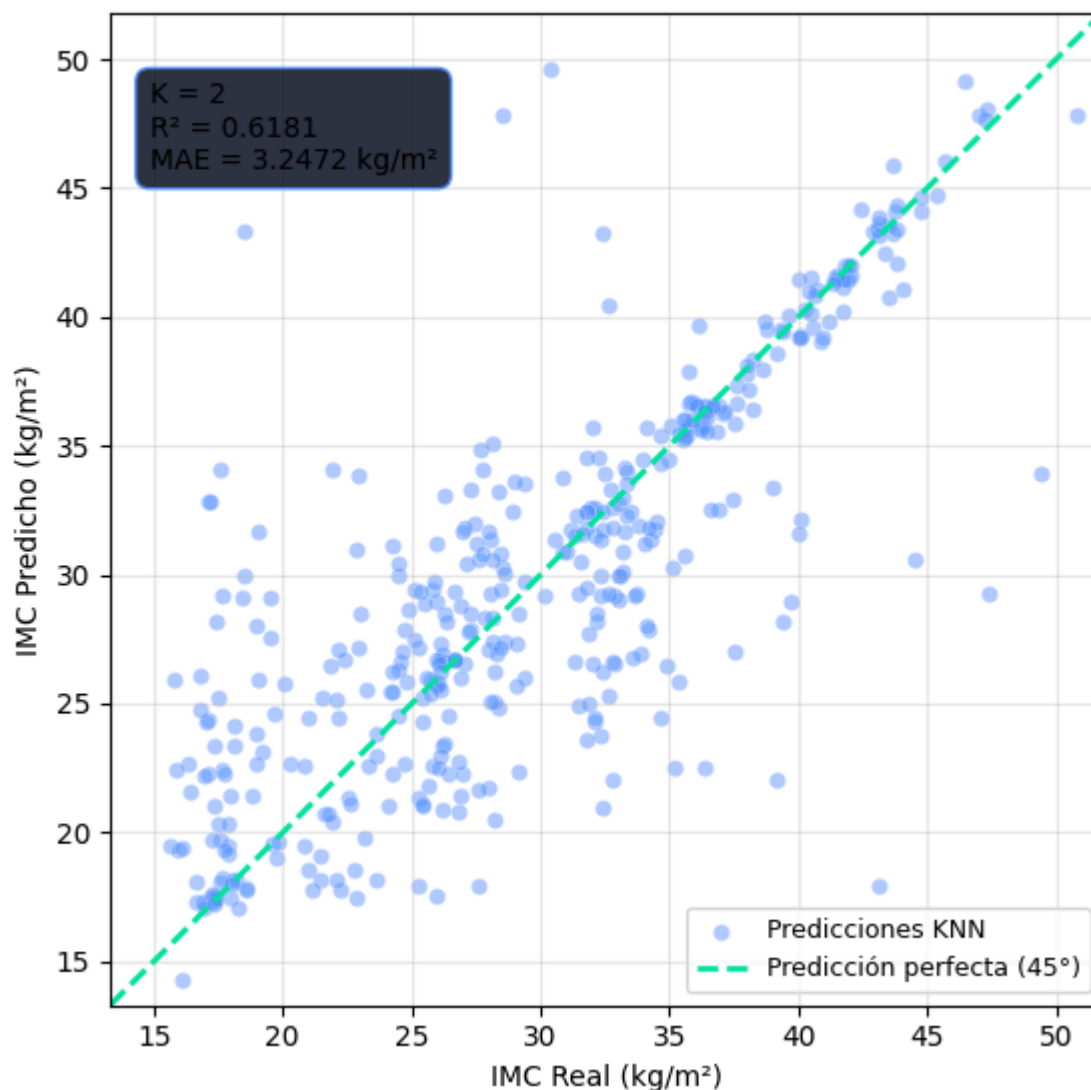
✓ Modelo KNN entrenado con $k = 2$.

```
=====
RESULTADOS – KNN Regressor
=====
K vecinos           : 2
R² Train            : 0.9842
R² Test             : 0.6181
Diferencia (Train-Test): 0.3660
MAE (Error Abs. Medio): 3.2472 kg/m²
=====
Overfitting         : Overfitting alto X – prueba un k mayor
```



✓ Gráfica guardada → knn_r2_train_vs_test.png

KNN Regressor (k=2) — Real vs. Predicho



✓ Gráfica guardada → knn_real_vs_predicho.png

RESUMEN — KNN Regressor

```
Modelo      : KNeighborsRegressor
K óptimo    : 2 (pesos por distancia)
Features    : ['Age', 'CH20', 'TUE', 'FAF', 'FCVC']
Train / Test: 80% / 20% (random_state=42)
R² (test)   : 0.6181
MAE (test)  : 3.2472 kg/m²
```

Para nuestro modelo de regresión no lineal, utilizamos KNN. Esta técnica consiste en predecir el IMC de un individuo basándose en la similitud con sus perfiles más cercanos en el conjunto de datos. Primero realizamos un proceso de optimización para encontrar el número de vecinos (valor K) que ofrezca el mejor equilibrio entre precisión y generalización.

Obtuvimos un K óptimo de 2. Esto significa que nuestro modelo solo tiene en cuenta a los 2 vecinos más cercanos. Este resultado nos advierte de un posible problema, ya que hace que el modelo sea altamente sensible a las variaciones locales de los datos. En nuestra gráfica de Valores Reales vs. Predicciones, comparamos el valor real del IMC frente a las estimaciones generadas por el modelo. Observamos que la mayoría de los puntos tienden a agruparse a lo largo de la línea diagonal (que representa la predicción perfecta), lo que significa que nuestro KNN logra estimar el IMC con buena exactitud. Las predicciones

que aparecen más dispersas indican la existencia de valores atípicos, detectados especialmente en rangos de IMC muy bajos.

Al profundizar en la evaluación de nuestro modelo KNN, decidimos comparar la precisión entre los datos con los que el modelo fue entrenado (Train) y los datos que intentó predecir por primera vez (Test). Observamos un R^2 casi perfecto de 0.98 en el entrenamiento frente a un 0.61 en la prueba. Esta drástica diferencia de 0.36 evidencia una presencia de sobreajuste (overfitting) alto. Esto sucede porque al utilizar un número tan bajo de vecinos ($K = 2$), el modelo se vuelve excesivamente sensible a las variaciones locales y al ruido; en la práctica, el algoritmo termina "memorizando" a sus dos vecinos más cercanos en el conjunto de entrenamiento en lugar de aprender una tendencia generalizable. A pesar de este fuerte sobreajuste, el algoritmo sigue siendo capaz de explicar el 61.8% de la variabilidad en casos nuevos, lo cual representa una mejora sustancial frente al 34% del modelo de regresión lineal. Además, logramos reducir el margen de error (MAE) a 3.24 kg/m². Sin embargo, este hallazgo analítico nos confirma que, para futuras optimizaciones, sería necesario probar con un valor de K mayor para penalizar esta excesiva sensibilidad y obligar al modelo a generalizar mejor la predicción del IMC.

A pesar de la mejora general observada, la inestabilidad detectada por un K tan bajo motivó la exploración de un modelo más robusto. En el apartado de Puntos Extra, se explica la implementación de la técnica de Random Forest, con la cual buscamos optimizar aún más la precisión y mitigar este sobreajuste.

Análisis de inferencia

Para finalizar nuestra exploración, debemos seleccionar un modelo para realizar un análisis de inferencia. Este modelo es el modelo de regresión lineal. Pese a obtener peores resultados que el KNN, este modelo nos permite interpretar directamente como cada variable afecta al IMC. El modelo KNN se basa en la memoria, donde se tiene en cuenta una comparación con la que no se puede conseguir una conclusión general del estudio, y menos con la k obtenida. Esto es fundamental con respecto a la problemática que se estudia en este proyecto, que es medir el impacto de los diferentes hábitos sobre el IMC. Observamos que "family_history" fue el predictor más grande, lo que permite inferir que la genética o el entorno familiar son determinantes críticos del IMC. La variable "SCC" mostró relación inversa con el IMC, lo que sugiere que la conciencia nutricional juega un papel importante, donde tiene una efectividad real en la gestión del peso corporal. Nuestro modelo presenta un margen de error de 5.24 kg/m², esto implica que nuestras inferencias tienen una incertidumbre de 5 puntos de IMC. Por lo tanto, nuestras conclusiones deberían tomarse como tendencias poblacionales y no como diagnósticos individuales exactos.

Finalmente, el alcance de este modelo es descriptivo y exploratorio. Una limitación fundamental es que el IMC no distingue entre masa grasa y masa muscular, lo que implica que el impacto de la gente que mide sus calorías a diario puede estar sesgado por individuos que ganan peso al entrenar y crecer el músculo. Además el aumento de heterocedasticidad implica que nuestro modelo es menos confiable para inferir comportamientos en personas con obesidad severa, donde las relaciones dejan de ser lineales.

Los resultados obtenidos indican que la obesidad no es el resultado de un solo hábito, sino de una red de factores biológicos e influenciados por la conducta de cada persona. La baja capacidad explicativa del modelo lineal ($R^2 = 0.34$) frente al éxito del modelo no lineal sugiere que las políticas de salud pública "estándar" (como las recomendaciones generales de ejercicio) pueden ser insuficientes si no consideran la individualidad y la complejidad de cada perfil. En el contexto real, este estudio revela que el automonitoreo, la historia familiar, el consumo de snacks entre comidas, la edad o el consumo de comidas altas en calorías son los pilares donde se debe enfocar la prevención. El hecho de que el

modelo falle más en rangos de obesidad altos (30-35 kg/m^2) nos dice que, una vez que se alcanza ese estado, la dinámica del peso cambia y requiere estrategias de intervención mucho más agresivas y especializadas.

Para la evolución de este trabajo, se podrían implementar comparaciones entre el IMC y el porcentaje de grasa, desarrollar un modelo específico que se centre en la obesidad severa (IMC = 30+) debido a la dificultad de nuestro modelo y así poder entender mejor porque el modelo lineal pierde precisión en esa zona. También profundizar en otras técnicas como el Random Forest que se analizará en el apartado de puntos extra puede ser útil para capturar las interacciones complejas que la regresión lineal no puede procesar.

Puntos extra

Como propuesta para superar las limitaciones de los dos modelos previos, se implementó la técnica Random Forest Regressor. Este es un algoritmo de aprendizaje por ensamble que construye múltiples árboles de decisión y combina sus resultados para una predicción más acertada. Nuestro modelo de entrenamiento utiliza un total de 100 árboles de decisión, donde se utilizaron las mismas variables seleccionadas para el modelo no lineal con información mutua. También se mantuvo la consistencia evaluando al modelo con un 20% de los datos que el algoritmo no conoció durante el entrenamiento.

```
In [12]: # =====
# MODELO DE REGRESIÓN NO LINEAL – Random Forest Regressor
# Objetivo : Predecir IMC (BMI) con hábitos de vida
# Variables : Age, CH20, TUE, FAF, FCVC (selección por MI)
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.feature_selection import mutual_info_regression

import warnings
warnings.filterwarnings('ignore')

# -----
# BLOQUE 1 • CARGA DEL DATASET
# -----

# Cambia la ruta por la tuya
df = pd.read_csv("Datos.csv")

# Calcular BMI si la columna no existe todavía
if 'IMC' not in df.columns:
    df['IMC'] = df['Weight'] / (df['Height'] ** 2)
    print("✓ Columna IMC calculada y añadida al dataframe.")

print(f"Dataset cargado: {df.shape[0]} filas × {df.shape[1]} columnas")
df.head()

# -----
```



```
# BLOQUE 2 · PREPROCESAMIENTO
```

```
#
```

```
# 2.1 Eliminar variables que causarían fuga de datos (data leakage)
# Weight y Height se excluyen porque el BMI se calcula con ellas.
cols_excluir = ['Weight', 'Height', 'NObeyesdad']
df_model = df.drop(columns=[c for c in cols_excluir if c in df.columns])
```

```
# 2.2 Codificar variables categóricas con LabelEncoder
```

```
le = LabelEncoder()
for col in df_model.select_dtypes(include='object').columns:
    df_model[col] = le.fit_transform(df_model[col].astype(str))
```

```
# 2.3 Separar target y features
```

```
y = df_model['IMC']
X_full = df_model.drop(columns=['IMC'])
```

```
# 2.4 Escalar con StandardScaler (los datos se escalan sobre X_full)
```

```
scaler = StandardScaler()
X_scaled = pd.DataFrame(
    scaler.fit_transform(X_full),
    columns=X_full.columns
)
```

```
# 2.5 Seleccionar las 5 variables elegidas por Información Mutua.
```

```
# Usamos .filter(like=) para capturar columnas con sufijos de
# dummificación (ej. 'Age', 'FCVC_1', etc.) si los hubiera.
FEATURES_MI = ['Age', 'CH20', 'TUE', 'FAF', 'FCVC']
```

```
# Función de selección flexible (nombre exacto o con sufijo)
```

```
def seleccionar_columnas(df, nombres):
    seleccionadas = []
    for nombre in nombres:
        coincidencias = [c for c in df.columns if c == nombre or c.startswith(nombre + '_')]
        seleccionadas.extend(coincidencias)
    return seleccionadas
```

```
cols_nolineal = seleccionar_columnas(X_scaled, FEATURES_MI)
X_nolineal = X_scaled[cols_nolineal]
```

```
print(f"\n✓ Variables del modelo No Lineal ({len(cols_nolineal)}):")
print(cols_nolineal)
```

```
#
```

```
# BLOQUE 3 · DIVISIÓN TRAIN / TEST
```

```
#
```

```
# 80 % entrenamiento – 20 % prueba
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_nolineal, y,
    test_size = 0.20,
    random_state = 42
)
```

```
print(f"\n✓ Tamaño de entrenamiento : {X_train.shape}")
print(f"✓ Tamaño de prueba : {X_test.shape}")
```

```
#
```

```
# BLOQUE 4 · ENTRENAMIENTO DEL MODELO
```

```
#
```

```

modelo_rf = RandomForestRegressor(
    n_estimators = 100,    # 100 árboles de decisión
    random_state = 42,    # reproducibilidad
    n_jobs       = -1     # usa todos los núcleos disponibles
)

modelo_rf.fit(X_train, y_train)
print("\n✓ Modelo entrenado correctamente.")

# -----
# BLOQUE 5 · MÉTRICAS DE EVALUACIÓN
# -----

y_pred_train = modelo_rf.predict(X_train)
y_pred       = modelo_rf.predict(X_test)

r2_train = r2_score(y_train, y_pred_train)
r2_test  = r2_score(y_test, y_pred)
mae      = mean_absolute_error(y_test, y_pred)
diferencia = r2_train - r2_test

print("\n" + "="*45)
print("  RESULTADOS – Random Forest Regressor")
print("="*45)
print(f"    R² Train           : {r2_train:.4f}")
print(f"    R² Test            : {r2_test:.4f}")
print(f"    Diferencia (Train-Test): {diferencia:.4f}")
print(f"    MAE (Error Abs. Medio): {mae:.4f} kg/m²")
print("="*45)

# Interpretación de overfitting
if diferencia < 0.05:
    estado = "Sin overfitting ✓ – el modelo generaliza bien"
elif diferencia < 0.15:
    estado = "Overfitting leve ⚠ – aceptable pero vigilar"
else:
    estado = "Overfitting alto ✗ – reducir profundidad o árboles"

print(f"    Overfitting           : {estado}")

# Gráfica comparativa R² Train vs Test
fig, ax = plt.subplots(figsize=(5, 4))

barras = ax.bar(
    ['R² Train', 'R² Test'],
    [r2_train, r2_test],
    color=['#00e5a0', '#4f8eff'],
    edgecolor='white', linewidth=0.5, width=0.4
)

for bar, val in zip(barras, [r2_train, r2_test]):
    ax.text(bar.get_x() + bar.get_width()/2, val + 0.01,
            f"{val:.4f}", ha='center', fontsize=11,
            fontweight='bold', color='white')

ax.set_ylim(0, 1.1)
ax.set_title("Comparativa R² – Train vs Test\nRandom Forest (IMC)",
            fontsize=12, fontweight='bold', pad=12)
ax.set_ylabel("R² Score", fontsize=10)
ax.axhline(y=1.0, color='white', linestyle=':', alpha=0.3)
ax.grid(axis='y', alpha=0.3)

```

```

plt.tight_layout()
plt.savefig("r2_train_vs_test.png", dpi=150, bbox_inches='tight')
plt.show()
print("✓ Gráfica guardada → r2_train_vs_test.png")

# Reasignar r2 para que los bloques siguientes lo usen correctamente
r2 = r2_test

# -----
# BLOQUE 6 · VISUALIZACIÓN 1 – Feature Importance
# -----

# Extraer importancias del modelo entrenado
importancias = pd.DataFrame({
    'Variable' : cols_nolineal,
    'Importancia': modelo_rf.feature_importances_
}).sort_values('Importancia', ascending=False).reset_index(drop=True)

# Paleta de colores degradado
colores = sns.color_palette("YlGn", len(importancias))[::-1]

fig, ax = plt.subplots(figsize=(8, 4))

sns.barplot(
    data = importancias,
    x = 'Importancia',
    y = 'Variable',
    palette = colores,
    ax = ax,
    orient = 'h'
)

# Etiquetas con el valor exacto al final de cada barra
for i, (val, _) in enumerate(zip(importancias['Importancia'], importancias['Variable'])):
    ax.text(val + 0.004, i, f"{val:.3f}", va='center', fontsize=10, color='white', fontweight='bold')

ax.set_title("Importancia de variables – Random Forest (IMC)", fontsize=13, fontweight='bold')
ax.set_xlabel("Importancia relativa", fontsize=10)
ax.set_ylabel("")
ax.set_xlim(0, importancias['Importancia'].max() + 0.08)
ax.axvline(x=1/len(importancias), color='salmon', linestyle='--', linewidth=1, label='Importancia media')
ax.legend(fontsize=9)
ax.grid(axis='x', alpha=0.4)

plt.tight_layout()
plt.savefig("feature_importance_rf.png", dpi=150, bbox_inches='tight')
plt.show()
print("✓ Gráfica guardada → feature_importance_rf.png")

# -----
# BLOQUE 7 · VISUALIZACIÓN 2 – Valores Reales vs. Predicciones
# -----

fig, ax = plt.subplots(figsize=(7, 6))

# Scatter: cada punto es una muestra del test set
ax.scatter(
    y_test, y_pred,
    alpha = 0.45,
    s = 35,

```

```

        color      = '#4f8eff',
        edgecolors= 'white',
        linewidths= 0.3,
        label      = 'Predicciones'
    )

# Línea de referencia perfecta (45°)
lim_min = min(y_test.min(), y_pred.min()) - 1
lim_max = max(y_test.max(), y_pred.max()) + 1
ax.plot([lim_min, lim_max], [lim_min, lim_max],
        color='#00e5a0', linewidth=2, linestyle='--', label='Predicción perfecta (45°)')

# Anotación de métricas dentro del gráfico
texto = f"R² = {r2:.4f}\nMAE = {mae:.4f} kg/m²"
ax.text(
    0.04, 0.92, texto,
    transform=ax.transAxes,
    fontsize=10, verticalalignment='top',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='#1a1d2e', edgecolor='#4f8eff', alpha=0.9)
)

ax.set_xlim(lim_min, lim_max)
ax.set_ylim(lim_min, lim_max)
ax.set_title("Valores Reales vs. Predicciones – BMI", fontsize=13, fontweight='bold', pad=12)
ax.set_xlabel("IMC Real (kg/m²)",      fontsize=10)
ax.set_ylabel("IMC Predicho (kg/m²)",  fontsize=10)
ax.legend(fontsize=9)
ax.grid(alpha=0.3)
ax.set_aspect('equal')

plt.tight_layout()
plt.savefig("reales_vs_predicciones_rf.png", dpi=150, bbox_inches='tight')
plt.show()
print("✓ Gráfica guardada → reales_vs_predicciones_rf.png")

# -----
# BLOQUE 8 • RESUMEN FINAL
# -----

print("\n" + "="*45)
print("  RESUMEN DEL PIPELINE")
print("="*45)
print(f"  Modelo      : RandomForestRegressor")
print(f"  Estimadores : 100 árboles")
print(f"  Features    : {cols_nolineal}")
print(f"  Train / Test: 80% / 20% (random_state=42)")
print(f"  R² (test)   : {r2:.4f}")
print(f"  MAE (test)  : {mae:.4f} kg/m²")
print("="*45)

```

✓ Columna IMC calculada y añadida al dataframe.

Dataset cargado: 2111 filas × 18 columnas

✓ Variables del modelo No Lineal (5):

['Age', 'CH20', 'TUE', 'FAF', 'FCVC']

✓ Tamaño de entrenamiento : (1688, 5)

✓ Tamaño de prueba : (423, 5)

✓ Modelo entrenado correctamente.

=====

RESULTADOS – Random Forest Regressor

=====

R² Train : 0.9505

R² Test : 0.6867

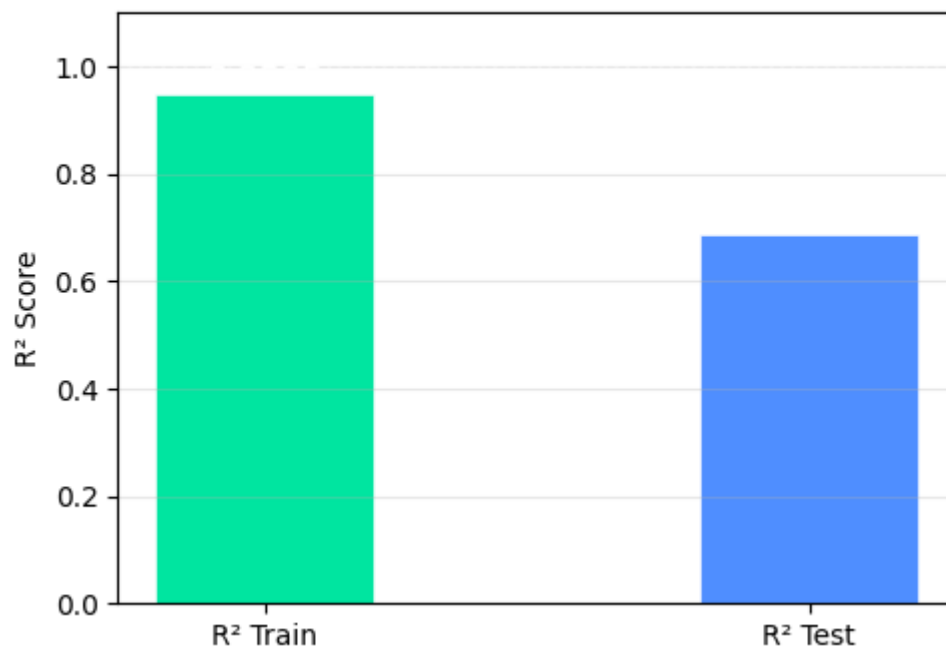
Diferencia (Train-Test): 0.2638

MAE (Error Abs. Medio): 3.1428 kg/m²

=====

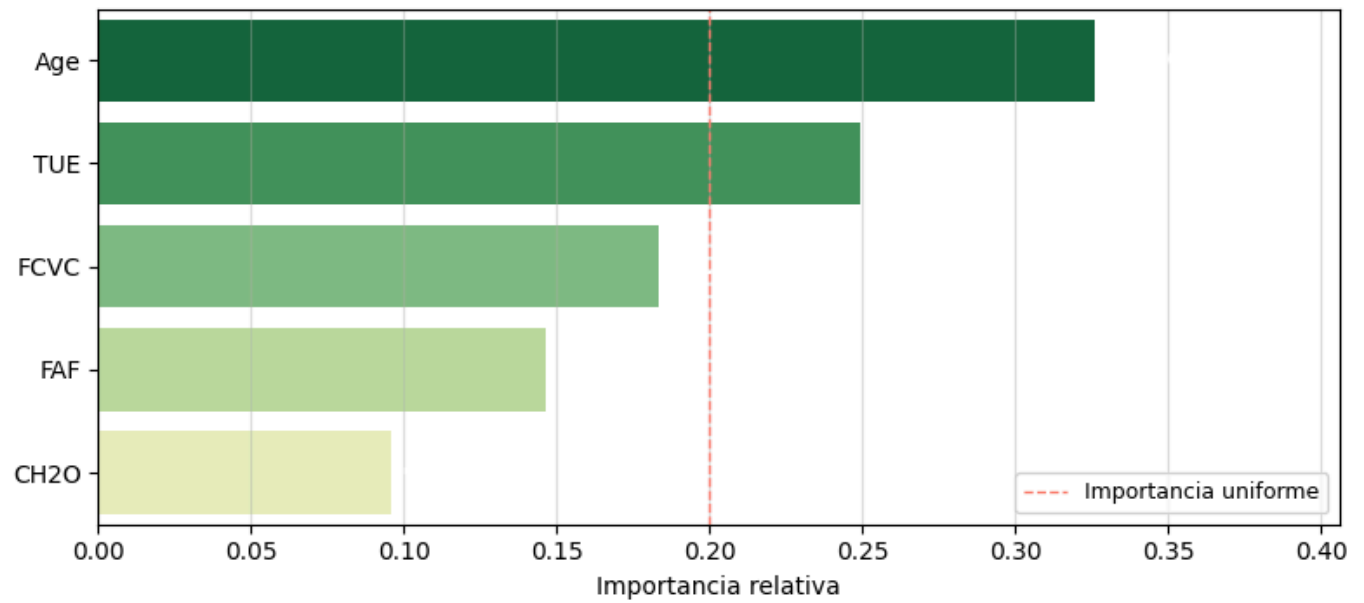
Overfitting : Overfitting alto X – reducir profundidad o árboles

Comparativa R² – Train vs Test Random Forest (IMC)



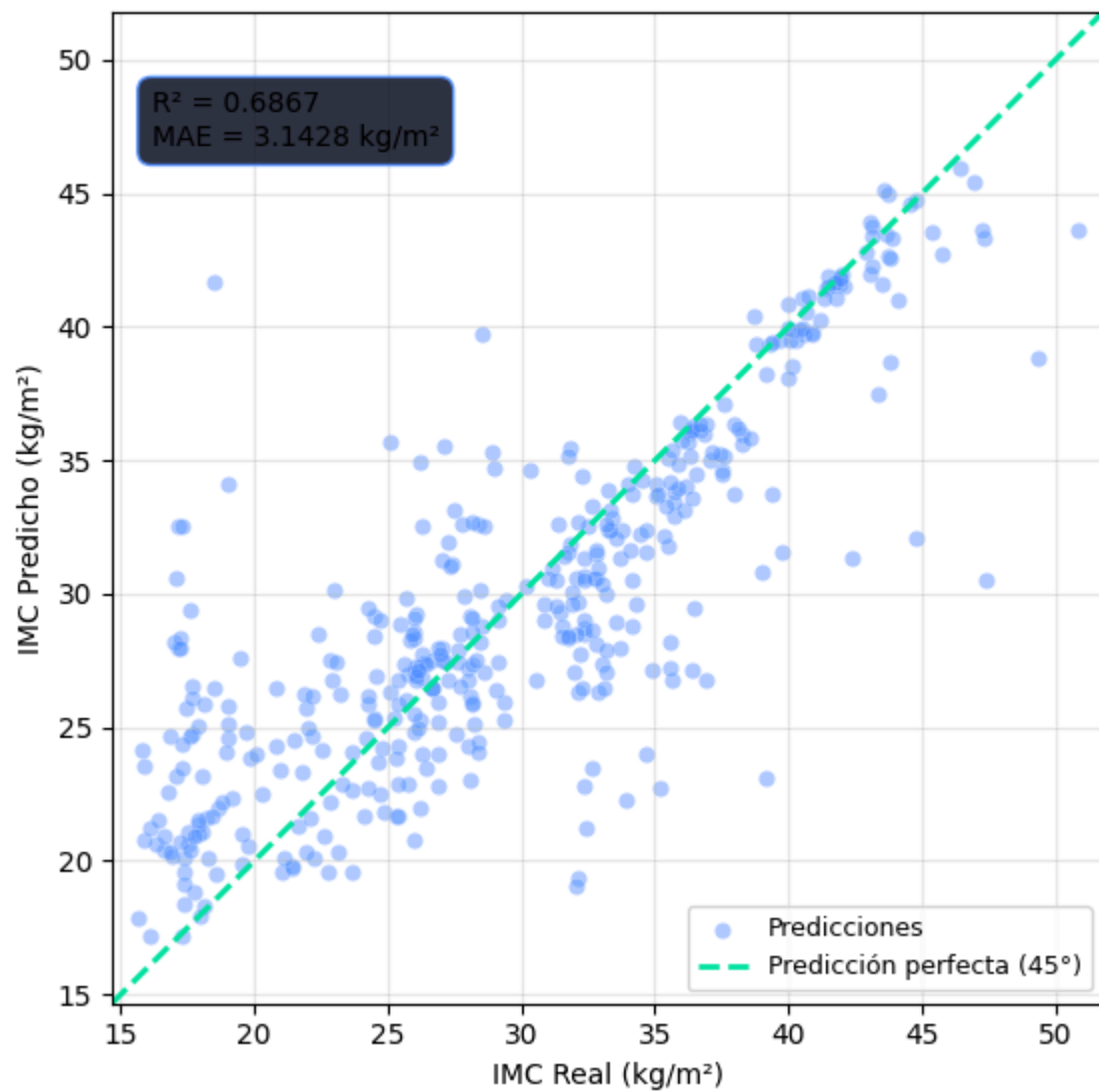
✓ Gráfica guardada → r2_train_vs_test.png

Importancia de variables — Random Forest (IMC)



✓ Gráfica guardada → feature_importance_rf.png

Valores Reales vs. Predicciones — BMI



RESUMEN DEL PIPELINE

```
Modelo      : RandomForestRegressor
Estimadores : 100 árboles
Features    : ['Age', 'CH2O', 'TUE', 'FAF', 'FCVC']
Train / Test: 80% / 20% (random_state=42)
R² (test)   : 0.6867
MAE (test)  : 3.1428 kg/m²
```

Los resultados obtenidos confirman que este modelo fue mejor que los dos anteriores. Al profundizar en la evaluación de nuestro Random Forest, decidimos comparar la precisión entre los datos con los que el modelo fue entrenado (Train) y los datos que intentó predecir por primera vez (Test). Observamos un R^2 de 0.95 en el entrenamiento frente a un 0.68 en la prueba. Esta notable diferencia (0.26) evidencia una clara presencia de sobreajuste (overfitting). Esto sucede porque el modelo, al ser una estructura tan compleja compuesta por múltiples árboles, termina "memorizando" los patrones específicos y el ruido de los datos de entrenamiento en lugar de aprender únicamente la regla general. A pesar de este sobreajuste, el algoritmo sigue siendo capaz de explicar el 68% de la variabilidad en casos nuevos, manteniéndose como nuestra herramienta predictiva más robusta, pero nos indica que en futuras optimizaciones será necesario limitar parámetros como la profundidad máxima de los árboles para obligar al modelo a generalizar mejor. Esto representa una mejora significativa con respecto a los resultados obtenidos de 0.34 en el modelo lineal, y 0.61 en el KNN. También se obtuvo el margen de error más bajo de todo el estudio, lo que significa que, en promedio, las predicciones del modelo fallan en solo 3 unidades de IMC, acercándose más a la realidad física de los usuarios. La primera gráfica realizada explica la importancia de cada variable, donde la edad es la variable más influyente por un margen amplio, superando la línea de importancia uniforme (línea verde), que representa el nivel de contribución esperado de la variable. Esto sugiere que el paso del tiempo es el predictor más importante. También debemos destacar el tiempo en pantallas (TUE), ya que aparece como el segundo factor más importante, que destaca el impacto del sedentarismo en el peso. Las demás variables se pueden considerar factores secundarios debido a que no pasan la línea de importancia uniforme, complementan el modelo con una relevancia relativamente menor. En la gráfica visual, observamos similitudes con la gráfica de KNN, aunque con Random Forest, logramos suavizar los errores y no se deja engañar tan fácilmente por casos aislados.

La implementación de este modelo demuestra que para hábitos multicausales, la combinación de múltiples árboles de decisión es superior a las técnicas simples. Mientras que la regresión lineal nos garantizó una mejor capacidad de inferencia (para entender las causas), el Random Forest se establece como nuestro mejor modelo de predicción, alcanzando un equilibrio óptimo entre precisión y estabilidad frente a datos nuevos.

Referencias

- Anthropic. (2024). Claude (Versión 3) [Modelo de lenguaje grande]. <https://claude.ai> (Nota: Úsala si utilizaste a Claude para redactar, estructurar o depurar alguna parte del código o texto).
- Google. (2024). Gemini (Versión 3.1 Pro) [Modelo de lenguaje grande]. <https://gemini.google.com> (Nota: Esta es la referencia oficial para citarme por la ayuda en la interpretación estadística, redacción y análisis de resultados).

- Martínez Torteya, A. (2026). SC3314 – Inteligencia Artificial [Material de clase]. Departamento de Ingeniería, Universidad de Monterrey.
- Palechor, F. M., & de la Hoz Manotas, A. (2019). Dataset for estimation of obesity levels based on eating habits and physical condition in individuals from Colombia, Peru and Mexico. UCI Machine Learning Repository [Alojado en Kaggle]. <https://doi.org/10.24432/C5H31Z> (Nota: Esta es la cita académica oficial de los creadores del dataset original del IMC y los hábitos que estás usando).
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830. <https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html> (Nota: Es casi obligatorio en ciencia de datos citar a Scikit-Learn si usaste sus algoritmos de LinearRegression, KNeighborsRegressor y RandomForestRegressor).
- Project Jupyter. (2023). Jupyter Notebook [Software computacional]. <https://jupyter.org/> (Nota: Referencia para el entorno de desarrollo donde ejecutaste todo tu código).